Grupo 12

GAMMA LAB 2.0

Juan Felipe Romero Ortiz
Laura Juliana Garzón Arias
Sebastián Almanza Galvis
Xiomara Herrera Suárez

PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS
BOGOTÁ, D.C.
2026

GAMMA LAB 2.0

Autores:

Juan Felipe Romero Ortiz
Laura Juliana Garzón Arias
Sebastián Almanza Galvis
Xiomara Herrera Suárez

MEMORIA DE PROYECTO DE GRADO REALIZADO PARA CUMPLIR UNO DE LOS REQUISITOS PARA EL TÍTULO EN INGENIERÍA DE SISTEMAS

Director

Ing. Cecile Eugine Gauthier Umaña

Jurados del Trabajo Final de Grado

Ing. Nombre del Primer Jurado
Ing. Nombre del Segundo Jurado

PONTIFICIA UNIVERSIDAD JAVERIANA

FACULTAD DE INGENIERÍA

PROGRAMA DE INGENIERÍA DE SISTEMAS

BOGOTÁ, D.C.

Noviembre, 2026

PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS

Rector de la Pontificia Universidad Javeriana

Nombre del Rector, S.J.

Decano de la Facultad de Ingeniería

Ing. Nombre del Decano de la Facultad

Director del Departamento de Ingeniería de Sistemas

Ing. Nombre del Director de Departamento, Ph.D.

Directora de Carrera de Ingeniería de Sistemas

Ing. Nombre de la Directora de Carrera

Artículo 23 de la Resolución No. 1 de Junio de 1946

"La Universidad no se hace responsable de los conceptos emitidos por sus alumnos en sus proyectos de grado. Sólo velará porque no se publique nada contrario al dogma y la moral católica y porque no contengan ataques o polémicas puramente personales. Antes bien, que se vean en ellos el anhelo de buscar la verdad y la Justicia"

AGRADECIMIENTOS

Juan Felipe Romero Ortiz:

[Escriba aquí los agradecimientos del primer autor. Puede usar varios párrafos; cada uno inicia con sangría automática.]

Laura Juliana Garzón Arias:

[Escriba aquí los agradecimientos del segundo autor.]

Sebastián Almanza Galvis:

[Escriba aquí los agradecimientos del tercer autor.]

Xiomara Herrera Suárez:

[Escriba aquí los agradecimientos del cuarto autor.]

Índice

1. INTRODUCCIÓN **1**

2. DESCRIPCIÓN GENERAL **2**
2.1. Oportunidad y problema . . . 2
2.1.1. Contexto del Problema . . . 2
2.1.2. Formulación del Problema . . . 3
2.1.3. Propuesta de Solución . . . 5
2.1.4. Justificación de la solución . . . 6
2.2. Descripción del Proyecto . . . 6
2.2.1. Objetivo General . . . 7
2.2.2. Objetivos Específicos . . . 7
2.3. Entregables, estándares utilizados y justificación . . . 7

3. CONTEXTO DEL PROYECTO **8**
3.1. Trasfondo . . . 8
3.2. Alternativas de solución . . . 10
3.2.1. Alternativa 1 . . . 10
3.2.2. Alternativa 2 . . . 10
3.2.3. Alternativa 3 . . . 10
3.2.4. Alternativa 4 . . . 11
3.3. Comparación de alternativas . . . 11

4. ANÁLISIS DEL PROBLEMA **12**
4.1. Requerimientos . . . 12
4.1.1. Requerimientos Funcionales . . . 12
4.1.2. Requerimientos No Funcionales . . . 13
4.2. Especificación Funcional . . . 14
4.3. Restricciones . . . 16
4.3.1. Restricciones Generales . . . 16
4.3.2. Restricciones de Software . . . 16
4.3.3. Restricciones de Hardware . . . 17

5. DISEÑO DE LA SOLUCIÓN **18**
5.1. Arquitectura . . . 18

5.2. Vistas de arquitectura 20
5.2.1. Vista lógica 20
5.2.2. Vista física 20
5.2.3. Vista de procesos 21
5.3. Diseño detallado 21
5.3.1. Estructura del sistema 21
5.3.2. Análisis de señales 28
5.3.3. Interfaz de usario (UI/UX) 28
5.4. Tecnologías 28

6. DESARROLLO DE LA SOLUCIÓN 29
6.1. Implementación 29
6.2. Producto 29
6.2.1. Funcionalidad 1 29
6.2.2. Funcionalidad 2 29
6.2.3. Funcionalidad 3 29

7. PRUEBAS DE DESARROLLO 30
7.1. Metodología aplicada 30
7.2. Entorno y herramientas 30
7.3. Organización y documentación 30

8. RESULTADOS 31
8.1. Resumen de pruebas unitarias 31
8.2. Observaciones y hallazgos 31
8.3. Resumen de pruebas comparativas contra el sistema de referencia 31
8.4. Observaciones y hallazgos 31
8.5. Resumen de pruebas de aceptación del usuario 31
8.6. Observaciones y hallazgos 31

9. CONCLUSIONES 32
9.1. Análisis de Impacto del Proyecto 32
9.1.1. A corto, mediano y largo plazo 32
9.1.2. Impacto en Ingeniería de Sistemas 32
9.1.3. Impacto en Contextos Globales, Económicos, Ambientales y Sociales 32
9.2. Conclusiones y Trabajo Futuro 32
9.2.1. Conclusiones 32

9.2.2. Trabajo futuro 32

10. REFERENCIAS **33**

11. ANEXOS **36**

11.1. MRCU - Matriz de Requerimientos y Casos de Uso 36
11.2. PMP — Project Management Plan 36
11.3. SAD - Software Architecture Document 36
11.4. SDD - Software Design Document 36
11.5. SRS - Software Requirements Specifications 36
11.6. TD - Test Document 36
11.7. MU - Manual de Usuario 36
11.8. VFP - Versión Final Propuesta de Proyecto 36
11.9. Aplicación de escritorio 36

1 INTRODUCCIÓN

[Escriba aquí el contenido de esta sección.]

2 DESCRIPCIÓN GENERAL

2.1 Oportunidad y problema

Las señales electroencefalográficas (EEG) son el registro de la actividad eléctrica generada por el cerebro, captada mediante electrodos. Esta actividad se puede representar mediante ondas cerebrales que se pueden estudiar en diferentes dominios de tiempo, frecuencia o tiempo-frecuencia, reflejando diferentes estados del individuo [1].

Una de las ventajas de estas señales es su alta resolución temporal, la cual permite observar cambios en la actividad neuronal en cuestión de milisegundos [2]. Esta característica convierte al EEG en una fuente de información valiosa para el estudio del sistema sensorial y motor, ya que permite seguir de cerca cómo el cerebro percibe su entorno y coordina el movimiento, procesos que ocurren y se transforman en fracciones de segundo [3]. Sin embargo, esta misma riqueza de información hace que las señales sean complejas: contienen una gran cantidad de datos que no son fáciles de interpretar a simple vista, por lo que se requieren herramientas capaces de organizarlas, limpiarlas y resaltar los patrones relevantes para su análisis.

En este escenario, un sistema computacional especializado en el tratamiento de estas señales permite potenciar dicho aprovechamiento, mejorando procesos de visualización, filtrado y extracción de características [4]. No obstante, alcanzar este nivel de procesamiento exige conocimientos avanzados de programación o el manejo de plataformas especializadas, una barrera técnica ampliamente documentada incluso en herramientas de código abierto orientadas a facilitar el trabajo del investigador [5]. Es en este contexto donde se enmarca Gamma Lab, una solución existente que, por tratarse de una primera iteración, presenta oportunidades de mejora en cuanto a funcionalidad y usabilidad. Sobre esta base, el presente trabajo desarrolla **la segunda versión** de dicha herramienta, ampliando sus funciones y corrigiendo las limitaciones identificadas en la primera iteración.

2.1.1 Contexto del Problema

GammaLab es una plataforma de software de código abierto creada en 2025. Su desarrollo respondió a una necesidad del Laboratorio de Electrofisiología de la Universidad Nacional de Colombia, cuyo flujo de trabajo dependía de herramientas fragmentadas y de licenciamiento comercial, lo que limitaba la reproducibilidad y el acceso de investigadores sin formación en programación. Frente a esto, se desarrolló un software libre, con arquitectura microkernel y ecosistema de plugins en Python, que permitiera el análisis de señales EEG en los dominios de tiempo, frecuencia y tiempo-frecuencia mediante una interfaz gráfica única

[6].

Como se expone en la memoria de trabajo de grado [6], Gamma Lab fue validada mediante pruebas comparativas frente al sistema de referencia BOARD-FTD-PACC [7], con un 87.5 % de coincidencia en los resultados, y mediante pruebas de aceptación con el usuario final, evaluando los 47 requerimientos funcionales definidos. De estos, 37 fueron aceptados sin reservas, 4 se aceptaron parcialmente y 6 quedaron fuera del alcance implementado; entre ellos la gestión de proyectos, la persistencia de sesiones de trabajo y el soporte a formatos adicionales de señal. A esto se suman las limitaciones que los propios usuarios señalaron en el control de la densidad de muestreo y en la selección de filtros avanzados [6]. Evidenciando que, si bien la primera versión validó la viabilidad de una alternativa libre y modular a las herramientas comerciales, dejó abiertas necesidades funcionales y de robustez que motivan el desarrollo de una segunda versión.

2.1.2 Formulación del Problema

Gamma Lab 1.0 [6] representó una solución inicial para la visualización y el procesamiento de señales EEG mediante una aplicación con interfaz gráfica. Sin embargo, propio de las limitaciones de una primera versión, se identificaron diversas brechas, entendidas tanto como deficiencias en funcionalidades existentes que requieren corrección como oportunidades de mejora asociadas a funcionalidades ausentes, o presentes de manera parcial y que aún no alcanzan un nivel de madurez adecuado. Las pruebas iniciales realizadas con usuarios del Laboratorio de Electrofisiología permitieron identificar estas brechas en los siguientes cuatro frentes principales:

- **Corrección de Errores Funcionales.** Se identificaron inconsistencias en diferentes módulos de procesamiento y análisis. Estas incluyen la ejecución manual de la remoción de artefactos sin un criterio de procesamiento definido, una implementación incorrecta de la normalización de la transformada Wavelet, fallas en la exportación de resultados hacia formatos CSV y JSON, ausencia de exportación en formato EMF, limitaciones en el cálculo de mediciones sobre múltiples ensayos y señales procesadas, y bloqueos de la aplicación durante la ejecución de los filtros elíptico y Chebyshev.

- **Fortalecimiento de la Experiencia de Navegación e Interacción.** A pesar del conocimiento técnico de los usuarios, la interfaz no comunica con claridad el estado del sistema ni los distintos modos de interacción disponibles, dificultando la navegación. Estas limitaciones se relacionan con la ausencia de identificadores de la señal, canal o análisis activo, la falta de retroalimentación visual sobre los elementos interactivos y los modos de selección, y la ausencia de ayudas contextuales para la configuración de

parámetros. En conjunto, estos aspectos dificultan la construcción de un modelo mental claro del funcionamiento de la aplicación para el usuario.

- **Estabilización de la Arquitectura y el Rendimiento del Sistema.** La revisión del kernel evidenció que el componente encargado de almacenar las señales y los resultados de análisis no ejecuta ninguna rutina de liberación de memoria cuando el usuario carga una nueva señal sin cerrar la aplicación, existe una función que registra la señal activa en el buffer interno, pero no una función complementaria que libere los datos de la sesión anterior. En consecuencia, el estado de cada carga se acumula indefinidamente en memoria durante la ejecución del programa, lo que deriva en un consumo creciente de recursos y en el bloqueo de la aplicación.

  Asimismo, la revisión del código confirmó que las funciones de procesamiento de la gran mayoría de los módulos de análisis se ejecutan de forma síncrona sobre el mismo hilo de la interfaz gráfica, sin que el núcleo del sistema provea un mecanismo compartido para delegar estos cálculos a procesos en segundo plano. Solo en casos puntuales se han implementado soluciones aisladas para evitar que la interfaz se congele durante un cálculo extenso por medio de un hilo de ejecución propio, pero no a nivel estructural de la herramienta. Esta dependencia de un único flujo de ejecución principal genera un cuello de botella que se agrava a medida que aumenta el tamaño de las señales o la complejidad de los análisis solicitados.

- **Ampliación del Alcance Analítico.** Las funcionalidades actuales de la herramienta se concentran en describir la amplitud, la distribución de energía por banda y la evolución temporal de la potencia de una misma oscilación. Sin embargo, estas técnicas no permiten caracterizar la relación entre distintos ritmos cerebrales que ocurren de forma simultánea sobre una misma señal. Esto representa una oportunidad para incorporar métodos de análisis complementarios orientados a cuantificar el acoplamiento entre componentes de distinta frecuencia de una misma señal, ampliando así la profundidad interpretativa que Gamma Lab puede ofrecer.

En conjunto, las brechas identificadas evidencian que la herramienta, aún no reúne las condiciones necesarias para consolidarse como una plataforma de uso sostenido dentro del Laboratorio de Electrofisiología. Esto plantea la necesidad de una segunda iteración orientada a fortalecer la plataforma sobre una base técnica que permita su evolución, adopción y utilización en el tiempo.

2.1.3 Propuesta de Solución

En respuesta a lo anterior, se plantea el desarrollo de Gamma Lab 2.0 como la segunda iteración de la plataforma, concebida para consolidar la base técnica establecida en la primera versión y dar continuidad a su evolución como herramienta para el procesamiento y análisis de señales EEG. La solución se organiza en torno a los mismos cuatro frentes descritos en la formulación del problema, de modo que cada brecha identificada cuenta con una medida concreta que la resuelve.

- **Corrección de Errores Funcionales en los Análisis y Procesos Existentes.** Este frente comprende la actualización de los módulos de procesamiento y análisis para garantizar un comportamiento consistente durante la ejecución de la herramienta. La propuesta incorpora la revisión del procedimiento de remoción de artefactos, la corrección de la normalización de la transformada Wavelet, la actualización de los mecanismos de exportación de resultados, la ampliación del módulo de mediciones para soportar múltiples ensayos y señales procesadas, la incorporación del formato de exportación EMF y la estabilización de los filtros digitales implementados.

- **Fortalecimiento de la Experiencia de Navegación e Interacción.** La solución redefine la interacción con la herramienta mediante una interfaz que comunica de forma explícita el contexto de trabajo del usuario. Para ello, incorpora elementos de contextualización de la señal, el canal y el análisis activo, indicadores visuales del estado de interacción, ayudas para la configuración de parámetros y una reorganización de la navegación que reduzca la dependencia de menús contextuales como mecanismo principal de operación. Adicionalmente, incorpora la persistencia de proyectos y sesiones de trabajo.

- **Estabilización de la Arquitectura y el Rendimiento del Sistema.** La propuesta fortalece el núcleo de la aplicación mediante mecanismos para la gestión del ciclo de vida de los recursos utilizados durante el procesamiento de señales y la revisión del esquema de ejecución de los módulos de análisis. Como parte de esta evolución, se plantea la incorporación de un mecanismo compartido de ejecución en segundo plano dentro del kernel como orquestador, que permita desacoplar el procesamiento de la interfaz gráfica y establecer una base común para soportar análisis de mayor demanda computacional.

- **Ampliación del Alcance Analítico de la Herramienta.** La segunda iteración incorpora un nuevo módulo para el análisis de *Phase-Amplitude Coupling* (PAC), ampliando las capacidades hacia el estudio de la interacción entre componentes de distinta frecuencia de una misma señal. El módulo se integra bajo la arquitectura de plugins de la plataforma,

permitiendo su incorporación como una capacidad analítica adicional dentro del flujo de procesamiento existente.

2.1.4 Justificación de la solución

La extensión de Gamma Lab busca consolidarla como una herramienta capaz de mantenerse y evolucionar en el tiempo. Las brechas identificadas afectan atributos de calidad del software, entre ellos la estabilidad, la extensibilidad, la usabilidad y la confiabilidad, de los cuales depende la adopción de esta como plataforma de investigación dentro del laboratorio. En particular, una arquitectura inestable compromete la escalabilidad prevista por el diseño de microkernel e impide extender la herramienta de forma consistente; una experiencia de navegación poco intuitiva reduce la accesibilidad que motivó el desarrollo de la interfaz gráfica como alternativa a herramientas basadas en programación; y la presencia de errores funcionales compromete la confiabilidad de los resultados obtenidos.

Además, es importante considerar que Gamma Lab está concebido para operar en un entorno con baja disponibilidad de mantenimiento por parte del equipo desarrollador, por lo que su utilización no puede depender de soporte continuo para interpretar la interfaz o resolver incidencias durante la ejecución. En estas condiciones, la herramienta debe ofrecer una interacción autónoma, resultados analíticos verificables y una infraestructura de software capaz de soportar su evolución de manera estable. Solo sobre esta base resulta pertinente ampliar su alcance analítico, permitiendo consolidar a Gamma Lab como una plataforma de software libre, sostenible y extensible para el procesamiento y análisis de señales EEG.

2.2 Descripción del Proyecto

Gamma Lab 2.0 corresponde a la segunda iteración de la plataforma de software libre para el procesamiento y análisis de señales EEG utilizada por el Laboratorio de Electrofisiología de la Universidad Nacional de Colombia. El proyecto abarca la corrección de las funcionalidades existentes, el fortalecimiento de la base técnica y arquitectónica de la herramienta, y la ampliación de su nivel de análisis, partiendo de la arquitectura microkernel y del ecosistema de plugins ya establecidos en la primera versión, y conservando su enfoque de interfaz gráfica como alternativa accesible frente a herramientas comerciales o basadas en programación. Con el propósito de transformar a Gamma Lab de un prototipo funcional en una plataforma consolidada, capaz de responder con solidez a las exigencias de la investigación en electrofisiología.

2.2.1 Objetivo General

Desarrollar GammaLab 2.0 como una versión extendida de la aplicación original que amplíe las capacidades de análisis de señales EEG e incorpore una arquitectura escalable, con el fin de mejorar el rendimiento del sistema y soportar un mayor número de análisis en los dominios de tiempo, frecuencia y tiempo-frecuencia.

2.2.2 Objetivos Específicos

1. Identificar brechas de mejora en la versión 1.0 Gamma Lab.
2. Diseñar el planteamiento estructural de la solución para abordar brechas detectadas.
3. Implementar la solución a través de software.
4. Probar la solución mediante validaciones del usuario.

2.3 Entregables, estándares utilizados y justificación

[Escriba aquí el contenido de esta sección.]

3 CONTEXTO DEL PROYECTO

3.1 Trasfondo

Gamma Lab 2.0 busca ampliar y estabilizar una herramienta que ya opera sobre un área concreta: la actividad eléctrica del cerebro registrada mediante *EEG*. Para comprender el alcance de esta ampliación, es necesario partir de la naturaleza de la señal que se analiza. Esta actividad es la manifestación macroscópica de la comunicación entre neuronas, que se transmiten información mediante impulsos eléctricos [8]. Cuando estos impulsos se sincronizan a gran escala, se organizan en oscilaciones que un EEG registra y que la comunidad científica clasifica en bandas de frecuencia estandarizadas [9]: Delta (0,5-4 Hz) y Theta (4-8 Hz), predominantes en estados de sueño profundo o de conciencia reducida [10]; Alpha (8-13 Hz), asociada a la relajación en vigilia [9]; Beta (13-30 Hz), vinculada a la atención activa [9]; y Gamma (mayor a 30 Hz), asociada a procesos de integración neuronal y cómputo cognitivo complejo [11]. Estas oscilaciones constituyen, por tanto, uno de los principales elementos sobre los que se estructura el análisis de señales EEG y, en particular, el procesamiento realizado por la herramienta.

Sin embargo, para estudiar estas oscilaciones no basta con analizar la señal desde una única perspectiva. Una misma señal EEG puede estudiarse desde diferentes dominios de análisis, en donde cada uno permite caracterizar un aspecto particular de su comportamiento:

- **Dominio del tiempo**: caracteriza la representación de la señal respecto al tiempo. Examina parámetros como su amplitud, duración y morfología, e incluye los Potenciales Relacionados con Eventos (ERP), respuestas cerebrales promediadas y sincronizadas con la ocurrencia de un estímulo [2].

- **Dominio de la frecuencia**: caracteriza la composición espectral de la señal. Mediante la Transformada de Fourier (*FFT*) descompone la señal en sus componentes de frecuencia, y mediante la Densidad Espectral de Potencia (*PSD*) cuantifica la contribución de cada banda de frecuencia [1, 4].

- **Dominio de tiempo-frecuencia**: caracteriza cómo evoluciona la composición espectral de la señal a lo largo del tiempo. Combina ambas perspectivas mediante la Transformada de Fourier de Tiempo Corto (*STFT*), que segmenta la señal en ventanas temporales sucesivas, o la Transformada de *Wavelet* (*WT*), que examina la señal en múltiples escalas simultáneamente, proporcionando mayor resolución temporal en las frecuencias altas y mayor resolución espectral en las bajas [4].

Estos tres dominios son, en conjunto, el lenguaje técnico sobre el que ya opera Gamma Lab y sobre el que se apoyan sus módulos de preprocesamiento, *wavelet* y promediado de ensayos.

Ahora bien, estos tres dominios comparten una limitación de fondo: analizan las bandas de oscilación de manera individual. Por ello, no permiten caracterizar directamente qué ocurre cuando dos bandas distintas se activan de forma simultánea sobre una misma señal y existe una relación entre ellas. Este tipo de interacción se conoce como acoplamiento cruzado de frecuencias (*Cross-Frequency Coupling*, CFC). Dentro de sus distintas formas, el acoplamiento fase-amplitud (*Phase-Amplitude Coupling*, PAC) describe una relación en la que la fase de una oscilación lenta modula la amplitud de una oscilación rápida; este fenómeno se ha asociado con procesos de comunicación, coordinación y cómputo neuronal a través de distintas escalas espaciales y temporales [12]. Su análisis requiere, por tanto, medidas específicas de acoplamiento, como el índice de modulación para relaciones dentro de un mismo canal [12] y la Consistencia de Fase por Pares (*Pairwise Phase Consistency*, PPC) para relaciones entre canales [13]. Estas medidas constituyen la base del nuevo módulo PAC que Gamma Lab 2.0 incorpora, tanto para el análisis de un solo canal como para el análisis inter-canal.

La incorporación de esta nueva capacidad analítica, sin reescribir lo que ya existe, se apoya en una decisión arquitectónica adoptada desde la primera versión de Gamma Lab. La herramienta se construyó sobre una arquitectura de *microkernel*, un patrón de diseño en el que un núcleo central concentra los servicios esenciales del sistema, mientras que cada dominio o técnica de análisis se implementa como un *plugin* independiente que puede incorporarse sin modificar el núcleo ni los demás módulos [14]. Esta estructura favorece la extensibilidad del sistema y permite incorporar nuevas técnicas de análisis, como PAC. Sin embargo, el crecimiento de los módulos también incrementa las exigencias sobre el núcleo, especialmente en la coordinación y gestión de los procesos de cómputo [14]. En su estado actual, la arquitectura se encuentra próxima a su límite de capacidad para sostener de manera estable las funcionalidades que ya integra, por lo que la incorporación de nuevas capacidades hace necesario fortalecer los mecanismos de coordinación y evolucionar el núcleo hacia un mecanismo de orquestación que permita gestionar de forma centralizada y eficiente el procesamiento de los distintos módulos de análisis.

En conjunto, estos elementos delimitan el trasfondo técnico y analítico de Gamma Lab 2.0. La ampliación hacia PAC y la estabilización del núcleo son los frentes que exigen esta fundamentación conceptual previa; la corrección de errores funcionales y el fortalecimiento de la experiencia de interacción completan el alcance de esta segunda iteración. A partir de esta base, es posible situar la propuesta frente a las soluciones existentes y reconocer las

aproximaciones que actualmente se emplean para el análisis de señales EEG.

3.2 Alternativas de solución

A continuación, se presentan cuatro herramientas ya existentes para el análisis y procesamiento de señales EEG, seleccionadas por abordar problemáticas similares a las que enfrenta Gamma Lab 2.0. El objetivo es identificar sus capacidades y limitaciones frente al alcance definido para esta segunda iteración, retomando el ejercicio comparativo realizado durante el desarrollo de Gamma Lab 1.0 [6].

3.2.1 Alternativa 1

EEGLAB es una caja de herramientas interactiva de MATLAB para el procesamiento de datos electrofisiológicos continuos y relacionados con eventos (EEG, MEG y otras modalidades), que incorpora análisis de componentes independientes (ICA), análisis en los dominios de tiempo y frecuencia, rechazo de artefactos, estadísticas relacionadas con eventos y distintos modos de visualización de datos promediados y de ensayos individuales [15]. Al operar sobre MATLAB, su uso requiere la licencia de este software, lo que limita su adopción en laboratorios con presupuesto restringido, además de exigir conocimientos avanzados de programación.

3.2.2 Alternativa 2

FieldTrip es una caja de herramientas desarrollada en MATLAB que ofrece un conjunto de funciones de alto nivel para el análisis de datos neurofisiológicos. A diferencia de otros entornos, no dispone de interfaz gráfica de usuario: las funciones se combinan dentro de un flujo de trabajo programado mediante un *script* en MATLAB que integra todas las etapas del análisis [16]. Esta característica hace que la herramienta resulte poco intuitiva para quienes se inician en la programación en MATLAB o en el análisis de señales EEG, MEG, iEEG o NIRS, a lo que se suma la misma barrera de licenciamiento que presenta EEGLAB.

3.2.3 Alternativa 3

Brainstorm es una aplicación colaborativa y de código abierto dedicada al análisis de grabaciones cerebrales (MEG, EEG, fNIRS, ECoG, electrodos de profundidad y electrofisiología multiunidad). Su uso es gratuito y no requiere licencia de MATLAB [17]. A través de su interfaz gráfica permite cubrir las necesidades de análisis sin necesidad de programar; sin embargo, dicha interfaz resulta poco intuitiva y su baja personalización introduce cierta rigidez en el flujo de trabajo.

3.2.4 Alternativa 4

BOARD-FTD-PACC es una interfaz gráfica de usuario desarrollada en MATLAB, diseñada para la visualización, cuantificación y análisis de registros neurofisiológicos. Proporciona herramientas personalizables orientadas al análisis de la actividad postsináptica y de datos oscilatorios neuronales complejos, principalmente mediante análisis cruzado de frecuencias [7]. Al ser el antecedente directo sobre el que se validó Gamma Lab 1.0, constituyó la base conceptual del proyecto; no obstante, su dependencia de una licencia de MATLAB y su interfaz poco intuitiva restringen su adopción en entornos con recursos limitados.

3.3 Comparación de alternativas

A continuación, se presenta una tabla comparativa que resume las características clave de las alternativas. Evaluando licenciamiento, costo, interfaz gráfica, independencia de MATLAB y conocimientos de programación requeridos; factores que determinan la accesibilidad y la viabilidad de adopción en un entorno de investigación con recursos limitados como el del Laboratorio de Electrofisiología.

Criterio	**EEGLAB**	**FieldTrip**	**Brainstorm**	**BOARD-FTD-PACC**	**GAMMA LAB 2.0**
Licencia gratuita	Parcial	Parcial	Parcial	Parcial	Sí
Costo anual (COP)	$3.948.000	$3.948.000	$0	$3.948.000	$0
GUI integrada	Parcial	No	Sí	Parcial	Sí
Independencia MATLAB	Si	Si	No	Si	No
Conocimientos en programación	Alto	Bajo	Alto	Bajo	Bajo

Tabla 1: Comparación de alternativas para análisis de señales neurofisiológicas

Como se evidencia en la tabla 1, Gamma Lab 2.0 consolida las ventajas de su versión anterior e incorpora mejoras clave frente a las alternativas existentes. Es la única alternativa completamente gratuita que además no requiere conocimientos avanzados de programación, eliminando así las dos barreras de acceso más comunes en laboratorios con recursos limitados. Gamma Lab 2.0 combina accesibilidad económica con una interfaz gráfica intuitiva, reduciendo significativamente la curva de aprendizaje para nuevos investigadores. BOARD-FTD-PACC, al ser el predecesor directo de Gamma Lab, sirvió como base conceptual del proyecto, pero requiere licencia de MATLAB, lo que limita su adopción en entornos con presupuestos restringidos, además de contar con una interfaz poco intuitiva que dificulta su uso para personas sin conocimientos en programación. Gamma Lab 2.0 aporta así al ecosistema de ciencia abierta una alternativa libre, accesible y personalizable para el análisis de señales neurofisiológicas, eliminando la barrera de los conocimientos en programación sin comprometer la profundidad analítica del software.

4 ANÁLISIS DEL PROBLEMA

4.1 Requerimientos

El levantamiento de requerimientos para Gamma Lab 2.0 partió de las brechas identificadas en la versión antecesora y de las necesidades expresadas por el Laboratorio de Electrofisiología. El conjunto resultante de este proceso se documentó bajo el estándar ISO/IEC/IEEE 29148 [18] y se clasificó en funcionales y no funcionales. En total se identificaron **81 requerimientos**, de los cuales **45 son funcionales y 36 no funcionales**. Las siguientes subsecciones presentan una explicación general de ambos grupos y los criterios bajo los cuales se organizaron; el detalle uno a uno se documenta en el anexo de Especificación de Requisitos de Software (SRS) (FALTA CITAR ANEXO AQUÍ) , y la información presentada a continuación se alinea con dicho documento.

4.1.1 Requerimientos Funcionales

Los requisitos funcionales especifican las acciones y comportamientos observables que Gamma Lab 2.0 debe ejecutar ante una entrada, evento o solicitud del usuario. Para facilitar su organización y trazabilidad, estos se agruparon en módulos de acuerdo con la capacidad que habilitan, considerando aspectos como el tipo de dato que manejan, la operación que realizan sobre la señal y el propósito que cumplen dentro del flujo de trabajo. De este proceso surgieron cuatro módulos: IO y UI, de carácter transversal y presentes a lo largo de los flujos de análisis, y WV y PAC, de carácter específico y vinculados a técnicas de análisis particulares. Esta organización refleja, a su vez, los frentes de trabajo definidos para la solución: bajo esta lógica, IO y UI buscan fortalecer la funcionalidad base y mejorar la experiencia de usuario; WV se enfoca en corregir y fortalecer el funcionamiento de una técnica ya incorporada en Gamma Lab; mientras que PAC amplía el alcance analítico de la herramienta mediante la incorporación de una nueva técnica de análisis.

- **Módulo IO – Entrada/Salida.** Opera sobre los datos que entran y salen de la herramienta: la señal en sus formatos de origen (ABF, EDF, MAT), el proyecto de trabajo (`.glab`) y los resultados de los análisis. Sus operaciones son de carga, persistencia y exportación. En esa medida, IO garantiza la persistencia de los resultados y del estado del proyecto, permitiendo al usuario retomar su trabajo sin pérdida de información y disponer de sus resultados fuera de la herramienta.

- **Módulo UI – Interacción general.** Opera sobre la señal ya cargada y los trials derivados de ella, es decir, sobre los datos con los que el usuario interactúa directamente.

Sus operaciones son de edición y navegación: recortar, ajustar el muestreo, remover artefactos, hacer zoom y seleccionar o promediar trials. En esa medida, UI otorga al usuario control preciso y visible sobre la señal, mejorando su experiencia de interacción con las capacidades ya existentes de la herramienta.

- **Módulo WV – Wavelet/Escalograma.** Opera sobre el resultado de una técnica ya presente en GammaLab: el escalograma generado por wavelet. Sus operaciones son de ajuste y presentación, entre las que se incluyen elegir la escala, normalizar el resultado y exportarlo en formato vectorial, sin intervenir en el cálculo del escalograma en sí. En esa medida, WV corrige y fortalece esta técnica existente, dándole al usuario mayor control sobre su interpretación visual.

- **Módulo PAC – Acoplamiento fase-amplitud.** Opera sobre una relación nueva entre los datos de la señal: el acoplamiento entre la fase de una banda lenta y la amplitud de una banda rápida, intra e inter-canal. Sus operaciones cubren todo el ciclo del análisis, desde la selección de bandas y canales hasta el cálculo del acoplamiento, la coherencia de fase y la exportación de resultados. En esa medida, PAC amplía el alcance analítico de GammaLab mediante una técnica completamente nueva, concentrando cerca de la mitad de los requisitos funcionales de esta iteración.

4.1.2 Requerimientos No Funcionales

Los requisitos no funcionales definen las condiciones de calidad que debe cumplir Gamma Lab 2.0 durante su operación. Para su clasificación se utiliza el modelo FURPS+ [19], que contempla las categorías de usabilidad (U), rendimiento (P), confiabilidad (R) y soporte (S). En esta iteración, estos requisitos se enfocan principalmente en estabilizar la arquitectura y mejorar el rendimiento del sistema, mediante la consolidación del núcleo y el orquestador de tareas. Además, complementan los objetivos orientados a la corrección de errores y al mejoramiento de la experiencia de usuario.

- **U – Usabilidad.** Agrupa los requisitos orientados a mejorar la experiencia de interacción del usuario con la herramienta: retroalimentación visual durante operaciones prolongadas, diferenciación de mensajes según su severidad, y accesibilidad de las funcionalidades desde la interfaz principal.

- **P – Rendimiento.** Establece los tiempos máximos de respuesta y límites de consumo de recursos para las operaciones del sistema: carga de señales, cómputo de análisis, exportación de resultados y liberación de memoria tras el cierre de un proyecto o módulo.

- **R – Confiabilidad.** Garantiza la integridad y consistencia del sistema durante su operación: sincronización del acceso a datos compartidos, liberación de recursos al finalizar procesos, y preservación de la señal y los trials originales frente a cualquier operación de preprocesamiento.

- **S – Soporte.** Incluye los requisitos que sostienen la arquitectura interna de la herramienta: el registro centralizado de servicios en el núcleo, la ejecución de cómputo pesado a través del orquestador de tareas, y la incorporación de nuevos plugins de análisis sin modificar los componentes existentes.

4.2 Especificación Funcional

A partir del levantamiento de requerimientos, se construyeron los casos de uso del sistema, tomando cada requisito como insumo para identificar el flujo de trabajo, el actor involucrado y la funcionalidad que respondía a la necesidad correspondiente. Los requerimientos afines en flujo y propósito se integraron en un mismo caso de uso, mientras que las interacciones independientes dieron lugar a casos de uso propios. De este proceso resultaron **23 casos de uso, cada uno con al menos un requisito asociado**, garantizando la cobertura total de los requerimientos; la asignación entre requisitos y casos de uso que se documenta en el anexo de especificación de casos de uso. (FALTA CITAR ANEXO AQUÍ) .

A partir de los casos de uso definidos se configura una visión integral del sistema, en la que las funcionalidades se articulan para representar las capacidades de Gamma Lab 2.0 y las relaciones entre los componentes que intervienen en su ejecución. Como se observa en el diagrama 1, el modelo contempla un único actor, el Investigador, quien desencadena las funcionalidades del sistema mediante sus solicitudes, sin que se representen actores externos o procesos autónomos como iniciadores de casos de uso. La interacción se concentra inicialmente en las funcionalidades de Interfaz y Usabilidad, desde las cuales se accede a las capacidades de la herramienta y se establece la comunicación con el Núcleo y Ciclo de Vida, que actúa como elemento central para gestionar el estado de la aplicación y coordinar la ejecución de los procesos. A partir de este núcleo se integran los subsistemas de Preprocesamiento, Análisis y Exportación, que soportan las diferentes etapas del flujo de trabajo con las señales. Esta lectura por componentes es la que guía la construcción de Gamma Lab 2.0: cada subsistema del diagrama 1 corresponde a un componente identificable dentro de la arquitectura del sistema.

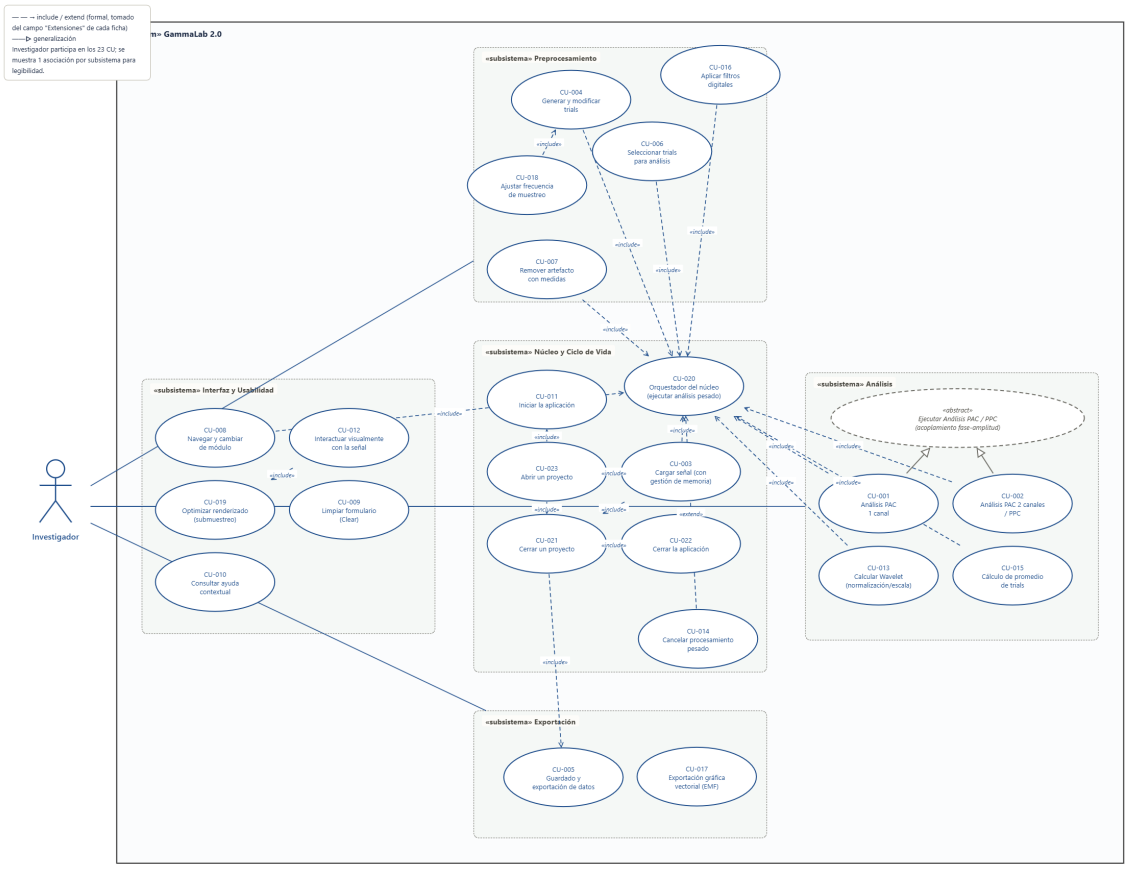


Figura 1: Diagrama de casos de uso del sistema Gamma Lab 2.0

Esta correspondencia entre subsistemas y componentes se detalla a continuación:

- **Subsistema de Núcleo y Ciclo de Vida.** Agrupa los casos de uso que gestionan el estado global de la aplicación: inicio, apertura y cierre de proyectos, carga de señal con gestión de memoria y cierre de la aplicación; junto con el orquestador del núcleo, encargado de despachar todo cómputo pesado y de habilitar su cancelación. Los casos de uso de Preprocesamiento y Análisis que implican cómputo intensivo se apoyan en este subsistema mediante relaciones de inclusión.

- **Subsistema de Interfaz y Usabilidad.** Agrupa los casos de uso que gobiernan la navegación entre módulos, la interacción visual con la señal (zoom, lectura de coordenadas, butterfly), el renderizado fluido de señales pesadas mediante submuestreo, la limpieza de formularios y la ayuda contextual.

- **Subsistema de Preprocesamiento.** Agrupa los casos de uso que transforman la señal

antes de cualquier análisis: la generación y modificación de trials, la selección de los trials que participan en un cálculo, la remoción de artefactos de estimulación con medidas de referencia previas al cambio, el ajuste de la frecuencia de muestreo y la aplicación de filtros digitales.

- **Subsistema de Análisis.** Agrupa las técnicas de procesamiento que constituyen el propósito analítico de la herramienta: el cálculo de la transformada Wavelet, el promedio de trials, y el análisis de acoplamiento fase-amplitud en su variante intra-canal (un canal) e inter-canal (dos canales y coherencia de fase, PPC), estas dos últimas modeladas como especializaciones de un caso de uso abstracto que comparte el procedimiento de cálculo.
- **Subsistema de Exportación.** Agrupa el guardado y exportación de datos del proyecto y la exportación gráfica en formato vectorial de los resultados analíticos.

4.3 Restricciones

Esta sección define los límites técnicos, normativos y operativos que condicionan el diseño, la implementación y el despliegue de Gamma Lab 2.0.

4.3.1 Restricciones Generales

El sistema opera íntegramente en inglés, tanto en la interfaz como en los mensajes de error y las alertas; ante fallos internos o errores del usuario, estos mensajes deben ser claros y no interrumpir el flujo de ejecución en curso. Asimismo, debe soportar la carga de archivos con múltiples canales y permitir ejecutar análisis sobre al menos uno de ellos, habilitando la selección de dos canales cuando el análisis sea de tipo inter-canal. Las gráficas, tablas y resultados numéricos que se generen deben poder exportarse desde la misma interfaz.

4.3.2 Restricciones de Software

El sistema mantiene una arquitectura modular orientada a plugins, en la que el Kernel orquesta la ejecución y permite incorporar nuevas capacidades sin modificar los plugins existentes. Se desarrolla en Python 3.11–3.12 y se distribuye como aplicación ejecutable localmente en Windows, aunque también puede ejecutarse directamente desde el código fuente del repositorio oficial.

El procesamiento de señales se apoya en NumPy, SciPy, MNE, TensorPAC y PyWavelets, mientras que la interfaz gráfica se construye con PySide6 y los componentes Qt; los formatos de entrada soportados son `.abf`, `.edf`, `.bdf` y `.mat`. La paralelización del Kernel, por su parte, se implementa mediante los mecanismos de hilos y *multiprocessing* incluidos en la biblioteca estándar de Python.

4.3.3 Restricciones de Hardware

El sistema debe ejecutarse en Windows 10 de 64 bits o superior, sobre un equipo que cumpla con los siguientes requerimientos mínimos:

- CPU de 4 núcleos a 2.0 GHz.
- 8 GB de RAM.
- 1 GB de VRAM.
- 3 GB de almacenamiento libre.

5 DISEÑO DE LA SOLUCIÓN

5.1 Arquitectura

Gamma Lab 2.0 toma como punto de partida la arquitectura de Gamma Lab 1.0 [6], conservando el estilo de microkernel y su implementación en Python. Esta decisión respondió, en parte, a que un cambio de estilo o de lenguaje habría obligado a reimplementar los dieciséis plugins existentes y a repetir su validación numérica, un costo que no se justificaba frente a los problemas identificados. Al mismo tiempo, conservar esta base permitió mantener la extensibilidad del sistema como mecanismo para incorporar nuevas técnicas de análisis. En consecuencia, la propuesta se orientó a fortalecer la arquitectura existente mediante la incorporación de mecanismos que permitieran mejorar la ejecución, la gestión de datos y la organización de los componentes, sin modificar el modelo general de extensibilidad.

La arquitectura de partida se organizaba alrededor de un componente `Kernel`, responsable de mantener el registro de servicios y plugins, y de distribuir eventos entre los módulos mediante señales de Qt. La carga dinámica de los plugins se realizaba a través de un gestor que identificaba sus archivos `properties.yml`, mientras que la interfaz `IPlugin` establecía las operaciones básicas para su integración con el núcleo y la interfaz gráfica. Sobre esta estructura se encontraban dieciséis plugins, organizados en funcionalidades de entrada y salida, preprocesamiento, medición y análisis en los dominios del tiempo, la frecuencia y el tiempo-frecuencia. La visualización se apoyaba en VTK, integrado en la interfaz desarrollada con PyQt5.

Tomando esta estructura como la base inicial, el diseño de Gamma Lab 2.0 consideró cinco necesidades arquitectónicas que orientaron la definición de la solución:

- La ejecución de los análisis debía independizarse del hilo principal de la interfaz. Para ello, se planteó un mecanismo centralizado de ejecución en segundo plano, con distribución del trabajo entre varios hilos y gestión del progreso, los errores y la cancelación de las tareas.

- La gestión de los datos debía garantizar un acceso controlado y una administración explícita de los recursos. En respuesta, se definió un esquema de almacenamiento compartido que contempla la sincronización de accesos, la propiedad de los datos y la liberación de memoria al finalizar su uso.

- Los servicios utilizados por distintos plugins debían mantener un estado común y evitar instancias independientes. Por esta razón, se estableció su centralización en el

núcleo, de modo que los diferentes módulos pudieran acceder a los mismos recursos y configuraciones.

- La lógica de cálculo debía separarse de los elementos encargados de la interfaz y la visualización. Esta decisión llevó a reorganizar los plugins para que las operaciones numéricas pudieran ejecutarse y validarse de manera independiente de su representación gráfica.
- La aplicación debía permitir conservar el contexto de trabajo y facilitar su ejecución en distintos entornos. Para ello, se incorporó el concepto de proyecto, junto con mecanismos de persistencia y recuperación de sesiones, gestión de rutas y distribución de la aplicación.

Sobre estos criterios se estructuró la solución, manteniendo los componentes centrales del microkernel y ampliando las capacidades de los módulos existentes. La arquitectura conserva los dieciséis plugins originales e incorpora tres nuevos: PAC, PPC y Modulation Index. Además, el diseño contempla una mayor cobertura de pruebas automatizadas, que pasó de 14 archivos en Gamma Lab 1.0 a aproximadamente 70 en Gamma Lab 2.0, incluyendo pruebas de concurrencia, cancelación, persistencia, liberación de memoria, instalación y rendimiento.

La Tabla 2 presenta una comparación de las principales características arquitectónicas y técnicas de Gamma Lab 1.0 y la solución diseñada para Gamma Lab 2.0.

Característica	Gamma Lab 1.0	Gamma Lab 2.0
Estilo arquitectónico	Microkernel con plugins	Microkernel con plugins y orquestador de tareas
Modelo de ejecución	Síncrono sobre el hilo de la interfaz; hilos aislados en dos plugins	Asíncrono mediante un servicio de tareas en el núcleo, con paralelización por trials
Progreso y cancelación	No disponibles	Centralizados en el orquestador
Almacén de datos	Diccionario sin sincronización ni liberación de memoria	Acceso sincronizado, propiedad por módulo y liberación de señales
Servicios compartidos	Instanciados de forma independiente en varios módulos	Instancia única registrada en el núcleo
Organización de los plugins	Cálculo y presentación en la misma clase	Funciones de cálculo independientes de la interfaz y la visualización
Persistencia	Inexistente	Proyectos `.glab` con esquema versionado
Ejecución y distribución	Scripts de arranque y rutas relativas	Instalador para Windows y resolución de rutas absolutas
Plugins	16	19 (PAC, PPC y Modulation Index)
Archivos de pruebas	14	$\approx$ 70 (concurrencia, cancelación, persistencia, memoria, instalación y rendimiento)

Tabla 2: Comparación arquitectónica entre Gamma Lab 1.0 y Gamma Lab 2.0

5.2 Vistas de arquitectura

[Escriba aquí el contenido de esta sección.]

5.2.1 Vista lógica

[Escriba aquí el contenido de esta sección.]

5.2.2 Vista física

[Escriba aquí el contenido de esta sección.]

5.2.3 Vista de procesos

[Escriba aquí el contenido de esta sección.]

5.3 Diseño detallado

[Escriba aquí el contenido de esta sección.]

5.3.1 Estructura del sistema

La Figura 2 muestra el diagrama de clases del sistema de Gamma Lab v.20 simplificado (Para más detalles, ver el documento Software Design Document (SDD)):

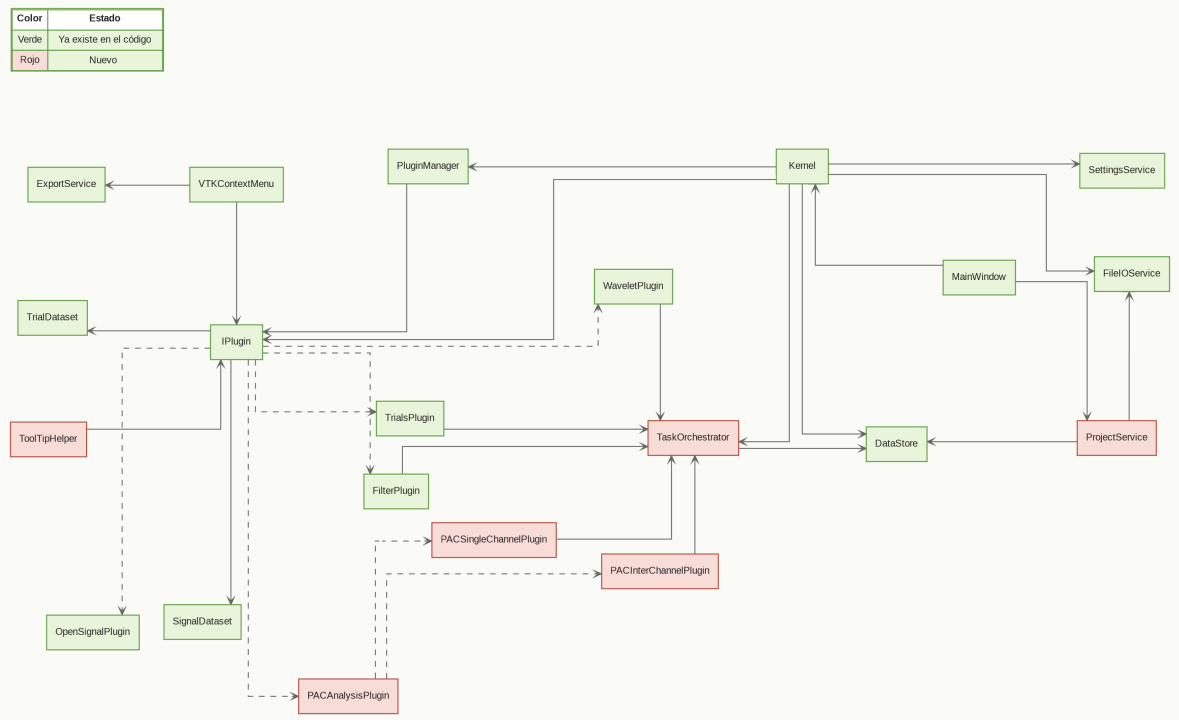


Figura 2: Diagrama de clases de Gamma Lab 2.0 simplificado

Mejoras del sistema

Núcleo y gestión de memoria: En Gamma Lab 1.0, el `DataStore` se encargaba de almacenar la señal activa, pero no contaba con un mecanismo para liberarla cuando dejaba de utilizarse. Esto hacía que, con cada nueva carga, los datos anteriores permanecieran en memoria, lo que podía provocar que la aplicación se bloqueara.

En Gamma Lab 2.0 se solucionó este problema mediante la incorporación del método `release_signals()`, que permite eliminar las señales anteriores y sus datos relacionados, como los trials, los conjuntos derivados, las mediciones y el trial en curso. Este proceso se ejecuta al cargar una nueva señal o al cerrar un proyecto.

Además, se implementó un procedimiento llamado `liberar_senal_anterior()`, que se encarga de realizar una limpieza más completa. Este procedimiento cancela las tareas que siguen pendientes, cierra las ventanas de VTK y libera los recursos utilizados por los widgets de las extensiones.

Por otra parte, se modificó el funcionamiento del `DataStore` para que solo el hilo de la interfaz gráfica pueda modificar su contenido. Si otro hilo intenta hacerlo, el sistema genera la excepción `EscrituraNoPermitida`. Tanto las operaciones de lectura como las de escritura utilizan un candado reentrante para controlar el acceso a los datos. De esta manera, se evita que varios procesos modifiquen la información al mismo tiempo y se reducen los posibles conflictos durante la ejecución de tareas en segundo plano.

Ejecución en segundo plano: En Gamma Lab 1.0, la mayoría de los análisis se ejecutaban directamente en el hilo de la interfaz gráfica. Algunas extensiones utilizaban sus propios hilos, pero no existía un mecanismo centralizado para controlar estas tareas.

En Gamma Lab 2.0 se incorporó `TaskService`, un servicio encargado de organizar y ejecutar las tareas del sistema. Este servicio utiliza una única cola de ejecución y procesa una tarea a la vez mediante un hilo de trabajo. Además, devuelve un objeto `TaskHandle`, que permite consultar el progreso de cada tarea y conocer si finalizó correctamente, presentó un error o fue cancelada, mediante las señales `progress`, `finished`, `failed` y `cancelled`.

La cancelación de tareas funciona de manera cooperativa, es decir, permite que cada tarea responda a la solicitud de cancelación. Si una tarea no responde después de dos segundos, el servicio deja de esperar su finalización y libera la interfaz para que el usuario pueda seguir interactuando con la aplicación.

También se agregó un mecanismo para evitar la acumulación de tareas. Cuando se agrega una nueva tarea de una misma extensión, las tareas anteriores que todavía están pendientes se descartan. Así, si el usuario presiona varias veces el botón *Generate*, no se generan múltiples cálculos innecesarios.

Como parte de esta mejora, las extensiones dejaron de crear sus propios hilos. Ahora todas las tareas pasan por `TaskService`, y los resultados de los cálculos se envían al hilo de la interfaz gráfica antes de mostrarse.

Durante el desarrollo también se evaluó la posibilidad de ejecutar los cálculos en procesos independientes. Sin embargo, esta alternativa se descartó porque las mediciones

mostraron que iniciar un proceso y enviarle los datos tomaba 1.486 ms, mientras que el cálculo en sí tardaba 1.163 ms. Es decir, el costo de preparar el proceso era mayor que el tiempo necesario para realizar el análisis.

En cambio, se optó por utilizar hilos, ya que las operaciones más exigentes de NumPy liberan el *GIL* y permiten aprovechar varios núcleos del procesador. En las pruebas de PPC, distribuir los trials entre cuatro hilos permitió alcanzar una aceleración de entre 1,8× y 2,7×, manteniendo los mismos resultados.

Rendimiento de visualización y cálculo: Se implementaron cuatro mejoras principales para reducir el consumo de memoria y mejorar el rendimiento de los análisis:

1. **Submuestreo de señales.** Se ajustó el cálculo del eje de frecuencias para utilizar la tasa de muestreo efectiva después del submuestreo, en lugar de utilizar la tasa solicitada inicialmente. Esto corrige un error de la versión anterior que podía generar desplazamientos de hasta un 11 % en el eje de frecuencias.

2. **Optimización del manejo de datos.** Los cálculos se realizan utilizando el formato `float64`, mientras que los datos enviados a VTK se convierten a `float32`. Esto permite mantener la precisión durante los cálculos y reducir el tamaño de los datos utilizados para la visualización.

3. **Procesamiento por bloques.** Las transformadas tiempo-frecuencia ahora se calculan por bloques de frecuencias, en lugar de procesar toda la matriz al mismo tiempo. Con este cambio, el consumo de memoria se redujo de 480 MB a 30 MB para una matriz de 500×60.000.

4. **Optimización de filtros.** Se incorporó un mecanismo de memoización para guardar el diseño de los filtros IIR y evitar repetir cálculos innecesarios. Esta mejora permitió reducir en 7,25 veces el tiempo necesario al cambiar de trial.

Corrección de los análisis existentes:

- **Filtros digitales:** Los bloqueos que se presentaban al utilizar los filtros elíptico y Chebyshev se debían a problemas de estabilidad numérica en los coeficientes con formato b/a, especialmente al trabajar con órdenes altos. Para solucionar esto, se modificó su implementación para utilizar secciones de segundo orden (SOS). También se agregó una validación para comprobar que los coeficientes sean valores finitos antes de utilizarlos. Finalmente, los filtros ahora devuelven copias de sus datos para evitar que otras partes del sistema modifiquen accidentalmente un filtro compartido.

- **Transformada Wavelet:** Se corrigió el método de normalización tiempo-frecuencia (`normalize_tf`), que ahora calcula la línea base de manera independiente para cada frecuencia. Además, se incorporaron tres métodos de normalización: *z-score*, *percent change* y *relative power*. También se ajustó la validación de los parámetros `fmin` y `fmax` para que tenga en cuenta la frecuencia de Nyquist y la tasa de muestreo efectiva.
- **Remoción de artefactos:** Se modificó el proceso de eliminación de artefactos para evitar que los cambios afecten directamente los trials originales. Ahora, cada modificación se realiza sobre un **conjunto derivado** mediante el método `crear_derivado`. Además, los cambios se guardan como una *receta*, que contiene el modo de corrección utilizado, los puntos seleccionados y los descartes realizados. Esto permite volver a aplicar el mismo procedimiento y obtener resultados reproducibles.
- **Exportación:** Se corrigieron los problemas que se presentaban al exportar tablas en formatos CSV y JSON. También se agregó la posibilidad de exportar resultados en formato vectorial EMF. Para ello, se desarrolló un escritor propio que genera los ejes, rótulos y líneas como elementos vectoriales, mientras que los mapas de calor se incorporan como imágenes. Por defecto, los archivos exportados se guardan en la carpeta `archivos/` del proyecto abierto.

Interfaz e interacción: Una de las decisiones técnicas relacionadas con la interfaz fue evitar que los avisos sobre los resultados de los cálculos abrieran ventanas modales. Esto se debe a que este tipo de ventanas genera un bucle de eventos adicional que podía provocar cierres inesperados cuando se seguían ejecutando los temporizadores de VTK. Por esta razón, se optó por utilizar avisos que no bloquean la interacción del usuario con la aplicación.

Nuevas funcionalidades

Proyectos y archivo `.glab`: En Gamma Lab 1.0 no existía una forma de guardar las sesiones de trabajo. Por esta razón, al cerrar la aplicación se perdía la información relacionada con los trials, el preprocesamiento, las mediciones y la configuración de los análisis.

Para solucionar esta limitación, Gamma Lab 2.0 incorpora `ProjectService` y un nuevo formato de proyecto llamado `.glab`. Cada proyecto se organiza en una carpeta que contiene tres elementos principales: un manifiesto en formato JSON con extensión `.glab`, una copia del archivo de señal original y una subcarpeta llamada `archivos/`, donde se almacenan las exportaciones. Al incluir una copia de la señal dentro del proyecto, es posible trasladarlo a otro equipo sin depender de la ubicación original del archivo.

Una de las principales decisiones de diseño fue evitar guardar directamente las matrices de datos dentro del manifiesto. En su lugar, se almacena la información necesaria para reconstruirlas cuando se vuelve a abrir el proyecto.

Por ejemplo, los trials se guardan mediante sus parámetros de recorte, los índices de los elementos descartados y las recetas de preprocesamiento aplicadas. Por su parte, los análisis almacenan los parámetros utilizados por cada extensión que tenga habilitada la opción `saves_analysis: true`. También se guarda la frecuencia de muestreo y su origen, lo que permite identificar si fue modificada manualmente.

Las mediciones son la excepción, ya que se almacenan completas. Esto se debe a que contienen pocos datos y dependen de puntos que el usuario marca manualmente.

Con esta estructura, los archivos de proyecto son más livianos, fáciles de consultar y cuentan con una versión de esquema mediante `schema_version`. Además, al reconstruir los datos utilizando el mismo código de procesamiento, se busca obtener los mismos resultados que se tenían antes de cerrar el proyecto.

El servicio también permite identificar si existen cambios sin guardar mediante el estado *dirty*. Para evitar problemas con los archivos, impide guardar proyectos directamente en carpetas del sistema, como Escritorio, Documentos o Descargas. Asimismo, evita copiar nuevamente la señal cuando esta ya se encuentra dentro del proyecto.

La copia de la señal se realiza por bloques y se guarda inicialmente en un archivo temporal. Una vez finalizada correctamente, este archivo se renombra para completar la operación. De esta manera, si el usuario cancela la copia, no queda un archivo incompleto. Finalmente, el sistema operativo puede abrir directamente los archivos `.glab` utilizando Gamma Lab.

Flujo general de la aplicación

Arranque: Cuando se inicia Gamma Lab, el archivo `main.py` se encarga de crear la aplicación Qt y mostrar la pantalla de bienvenida. Después, se crea el `Kernel` y se registran los servicios principales: `DataStore`, `FileIO` y `TaskService`.

Este registro se realiza antes de cargar las extensiones, ya que estas pueden necesitar acceder a dichos servicios desde su método `initialize`.

Una vez registrados los servicios, `PluginManager` recorre la carpeta `plugins/`, crea las instancias de cada extensión y las registra en el Kernel. Finalmente, se construye `MainWindow`, que organiza las extensiones registradas en la barra de categorías de la interfaz y se registra como un servicio adicional.

Si Gamma Lab se inicia directamente desde un archivo `.glab`, el sistema también se

encarga de cargar el proyecto correspondiente.

Carga de señal y apertura de proyecto: El usuario puede abrir archivos de señal en formatos `.abf`, `.edf` y `.mat`, o cargar un proyecto completo en formato `.glab`.

Cuando se abre un proyecto, `MainWindow` primero verifica si existen cambios sin guardar o cálculos que todavía se estén ejecutando. Después, solicita a `TaskService` que realice el proceso de preparación mediante `preparar_apertura()`.

Durante esta etapa, un hilo de trabajo se encarga de leer el manifiesto y el archivo de señal mediante `FileIOService`, sin modificar todavía el `DataStore` ni la interfaz gráfica.

Solo cuando la lectura finaliza correctamente se libera el proyecto anterior y se ejecuta el método `open()`. Este proceso se realiza desde el hilo de la interfaz y se encarga de registrar la nueva señal como activa, aplicar la frecuencia de muestreo corregida, reconstruir los trials a partir de sus parámetros de recorte y recuperar los descartes y las recetas de preprocesamiento sobre el conjunto derivado.

El orden de estos pasos es importante, ya que cada receta conserva los descartes que estaban vigentes cuando fue aplicada. Finalmente, se restauran las mediciones y se envían los parámetros guardados a las extensiones correspondientes.

Si la lectura del proyecto falla o el usuario cancela la operación, el proyecto que estaba abierto anteriormente se mantiene sin cambios.

Trabajo y análisis: Una vez cargada la señal, el usuario puede generar trials, realizar tareas de preprocesamiento, como filtrado, eliminación de artefactos y ajuste de la frecuencia de muestreo, y ejecutar los diferentes análisis disponibles.

Antes de iniciar cada análisis, el sistema valida los parámetros ingresados y cancela cualquier cálculo anterior de la misma extensión. Luego, envía la nueva tarea a `TaskService` para su ejecución en segundo plano.

Cuando el cálculo termina correctamente, el resultado se envía al hilo de la interfaz y se muestra mediante VTK. Además, el proyecto se marca como modificado para indicar que existen cambios que pueden guardarse.

Los resultados numéricos permanecen almacenados en la extensión que realizó el análisis. En cambio, el `DataStore` conserva únicamente la información compartida entre los componentes, como las señales, la señal activa, las mediciones y el trial en curso.

Guardado: El proceso de guardado comienza con el método `iniciar_guardado()`, que verifica que exista una señal activa y determina la carpeta donde se almacenará el proyecto.

Esta puede ser una ubicación utilizada anteriormente o una nueva carpeta seleccionada y validada por el usuario.

Si es necesario copiar el archivo de señal, esta operación se realiza mediante `TaskService`, que permite consultar el progreso y cancelar el proceso si es necesario.

Cuando la copia termina correctamente, se ejecuta `completar_guardado()`, que genera el manifiesto consultando la información de los trials, las mediciones y los parámetros almacenados en cada extensión. Al finalizar, el estado del proyecto cambia a `dirty = False`, indicando que todos los cambios han sido guardados.

Si la copia falla o se cancela, el sistema revierte el estado de la operación y evita generar un archivo de proyecto `.glab` incompleto.

Cierre: Al cerrar un proyecto o la ventana principal, el sistema utiliza el mismo mecanismo de confirmación para evitar la pérdida de información.

Si hay un cálculo en ejecución, se pregunta al usuario si desea continuar con el cierre, lo que implica cancelar las tareas pendientes. Si existen cambios sin guardar, se muestran las opciones *Save*, *Don't Save* y *Cancel*, para que el usuario decida cómo proceder.

Una vez autorizado el cierre, `ProjectService.close()` limpia el estado del proyecto y `liberar_senal_anterior()` se encarga de cancelar las tareas, cerrar los recursos de VTK, destruir los widgets, desvincular las extensiones de la señal y liberar los datos almacenados en el `DataStore`.

Después de este proceso, la aplicación regresa a la pantalla de inicio y libera la memoria utilizada para que pueda cargarse una nueva señal. Cuando se cierra completamente la aplicación, también se ejecuta el método `stop()` de cada extensión.

Aspecto	Gamma Lab 1.0	Gamma Lab 2.0
Persistencia	No permitía guardar proyectos	Incorpora proyectos `.glab` que pueden reconstruirse y trasladarse entre equipos
Memoria	Las señales anteriores permanecían almacenadas	Incorpora `release_signals()` para liberar datos al cambiar de señal o cerrar un proyecto
Ejecución	Los análisis se ejecutaban principalmente en el hilo de la interfaz	Incorpora `TaskService` para gestionar tareas, consultar su progreso y cancelarlas
Concurrencia de datos	No existía un control centralizado	El `DataStore` utiliza un único escritor y un mecanismo de bloqueo
Filtros IIR	Presentaba bloqueos con los filtros elíptico y Chebyshev	Utiliza el formato SOS con validaciones y memoización
Wavelet	Presentaba problemas en la normalización	Incorpora una línea base por frecuencia y utiliza la tasa de muestreo efectiva
Artefactos	Los cambios se aplicaban directamente sobre los datos originales	Utiliza conjuntos derivados y recetas para reproducir los cambios
Exportación	Presentaba fallas en CSV y JSON y no ofrecía exportación EMF	Corrige la exportación CSV y JSON e incorpora el formato vectorial EMF
Análisis	Incluía análisis en tiempo, frecuencia y tiempo-frecuencia	Incorpora PAC (en uno y dos canales) y PPC

Tabla 3: Principales mejoras de Gamma Lab 1.0 a Gamma Lab 2.0

5.3.2 Análisis de señales

[Escriba aquí el contenido de esta sección.]

5.3.3 Interfaz de usario (UI/UX)

[Escriba aquí el contenido de esta sección.]

5.4 Tecnologías

[Escriba aquí el contenido de esta sección.]

6 DESARROLLO DE LA SOLUCIÓN

6.1 Implementación

[Escriba aquí el contenido de esta sección.]

6.2 Producto

[Escriba aquí el contenido de esta sección.]

6.2.1 Funcionalidad 1

[Escriba aquí el contenido de esta sección.]

6.2.2 Funcionalidad 2

[Escriba aquí el contenido de esta sección.]

6.2.3 Funcionalidad 3

[Escriba aquí el contenido de esta sección.]

7 PRUEBAS DE DESARROLLO

7.1 Metodología aplicada

[Escriba aquí el contenido de esta sección.]

7.2 Entorno y herramientas

[Escriba aquí el contenido de esta sección.]

7.3 Organización y documentación

[Escriba aquí el contenido de esta sección.]

8 RESULTADOS

8.1 Resumen de pruebas unitarias

[Escriba aquí el contenido de esta sección.]

8.2 Observaciones y hallazgos

[Escriba aquí el contenido de esta sección.]

8.3 Resumen de pruebas comparativas contra el sistema de referencia

[Escriba aquí el contenido de esta sección.]

8.4 Observaciones y hallazgos

[Escriba aquí el contenido de esta sección.]

8.5 Resumen de pruebas de aceptación del usuario

[Escriba aquí el contenido de esta sección.]

8.6 Observaciones y hallazgos

[Escriba aquí el contenido de esta sección.]

9 CONCLUSIONES

9.1 Análisis de Impacto del Proyecto

[Escriba aquí el contenido de esta sección.]

9.1.1 A corto, mediano y largo plazo

[Escriba aquí el contenido de esta sección.]

9.1.2 Impacto en Ingeniería de Sistemas

[Escriba aquí el contenido de esta sección.]

9.1.3 Impacto en Contextos Globales, Económicos, Ambientales y Sociales

[Escriba aquí el contenido de esta sección.]

9.2 Conclusiones y Trabajo Futuro

[Escriba aquí el contenido de esta sección.]

9.2.1 Conclusiones

[Escriba aquí el contenido de esta sección.]

9.2.2 Trabajo futuro

[Escriba aquí el contenido de esta sección.]

10 REFERENCIAS

Referencias

[1] A. K. Singh y S. Krishnan, "Trends in EEG signal feature extraction applications," *Frontiers in Artificial Intelligence*, vol. 5, art. 1072801, ene. 2023, doi: 10.3389/frai.2022.1072801. [En línea]. Disponible: https://pmc.ncbi.nlm.nih.gov/articles/PMC9905640/.

[2] P. Walla, "The Power of Time: Editorial on the Advantages of Electroencephalography (EEG) and Event-Related Potentials (ERPs) in Affective and Cognitive Neuroscience," *Brain Sciences*, vol. 15, n.º 10, art. 1054, sep. 2025, issn: 2076-3425, doi: 10.3390/brainsci15101054. [En línea]. Disponible: https://www.mdpi.com/2076-3425/15/10/1054.

[3] K. Jayalaksshme Srinivasan, P. Periasamy, y S. Gunasekaran, "Motor Imagery and Motor Execution: A Narrative Review of Electroencephalographic (EEG) Signatures, Methodological Consistency, and Translational Applications," *Cureus*, vol. 17, n.º 9, art. e93011, sep. 2025, doi: 10.7759/cureus.93011. [En línea]. Disponible: https://pmc.ncbi.nlm.nih.gov/articles/PMC12550537/.

[4] A. Chaddad, Y. Wu, R. Kateb, y A. Bouridane, "Electroencephalography Signal Processing: A Comprehensive Review and Analysis of Methods and Techniques," *Sensors*, vol. 23, n.º 14, art. 6434, ene. 2023, issn: 1424-8220, doi: 10.3390/s23146434. [En línea]. Disponible: https://www.mdpi.com/1424-8220/23/14/6434 (Visitado: 28-04-2026).

[5] M. W. J. van Es, C. Gohil, A. J. Quinn, y M. W. Woolrich, "osl-ephys: a Python toolbox for the analysis of electrophysiology data," *Frontiers in Neuroscience*, vol. 19, art. 1522675, feb. 2025, doi: 10.3389/fnins.2025.1522675. [En línea]. Disponible: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2025.1522675/full.

[6] S. Asencio Rodríguez, M. F. González Valencia, J. E. Martínez Clavijo, y G. V. Villalobos Rocha, "Gamma Lab," Memoria de trabajo de grado, Programa de Ingeniería de Sistemas, Facultad de Ingeniería, Pontificia Universidad Javeriana, Bogotá, D.C., Colombia, nov. 2025.

[7] C. Gauthier-Umaña, M. Valderrama, A. Múnera, y M. O. Nava-Mesa, "BOARD-FTD-PACC: a graphical user interface for the synaptic and cross-frequency analysis derived

from neural signals," *Brain Informatics*, vol. 10, art. 12, may. 2023, issn: 2198-4026, doi: 10.1186/s40708-023-00191-x. [En línea]. Disponible: https://link.springer.com/article/10.1186/s40708-023-00191-x.

[8] P. Morris y O. Calaway, "3.1 Foundations of Neural Communication,.en *General Psychology*, ago. 2025. [En línea]. Disponible: https://acd.pressbooks.pub/generalpsychology/chapter/2192/.

[9] P. L. Nunez, .Electroencephalography (EEG),"*Reference Module in Neuroscience and Biobehavioral Psychology*, Elsevier, 2017, isbn: 9780128093245, doi: 10.1016/B978-0-12-809324-5.03049-2. [En línea]. Disponible: https://linkinghub.elsevier.com/retrieve/pii/B9780128093245030492.

[10] J. D. Kropotov, "Slow, Infra-Slow Potentials, and Delta Rhythms,.en *Quantitative EEG, Event-Related Potentials and Neurotherapy*, Elsevier, 2009, pp. 11-28, isbn: 9780123745125, doi: 10.1016/B978-0-12-374512-5.00001-3. [En línea]. Disponible: https://linkinghub.elsevier.com/retrieve/pii/B9780123745125000013.

[11] "Normal EEG variants,.en *Handbook of Clinical Neurology*, vol. 160, Elsevier, 2019, pp. 143-160, isbn: 9780444640321, doi: 10.1016/B978-0-444-64032-1.00009-6. [En línea]. Disponible: https://linkinghub.elsevier.com/retrieve/pii/B9780444640321000096.

[12] E. Combrisson, T. Nest, A. Brovelli, R. A. A. Ince, J. L. P. Soto, A. Guillot, y K. Jerbi, "Tensorpac: An open-source Python toolbox for tensor-based phase-amplitude coupling measurement in electrophysiological brain signals,"*PLOS Computational Biology*, vol. 16, n.º 10, art. e1008302, oct. 2020, issn: 1553-7358, doi: 10.1371/journal.pcbi.1008302. [En línea]. Disponible: https://doi.org/10.1371/journal.pcbi.1008302.

[13] M. Vinck, M. van Wingerden, T. Womelsdorf, P. Fries, y C. M. A. Pennartz, "The pairwise phase consistency: a bias-free measure of rhythmic neuronal synchronization,"*NeuroImage*, vol. 51, n.º 1, pp. 112-122, may. 2010, issn: 1053-8119, doi: 10.1016/j.neuroimage.2010.01.073. [En línea]. Disponible: https://doi.org/10.1016/j.neuroimage.2010.01.073.

[14] M. Richards, *Software Architecture Patterns*, 2.ª ed. Sebastopol, CA, EE. UU.: O'Reilly Media, 2022. [En línea]. Disponible: https://www.oreilly.com/library/view/software-architecture-patterns/9781098134280/.

[15] EEGLAB. [En línea]. Disponible: https://sccn.ucsd.edu/eeglab/index.php.

[16] Getting started, FieldTrip. [En línea]. Disponible: https://www.fieldtriptoolbox.org/getting_started/.

[17] Introducción, Brainstorm. [En línea]. Disponible: https://neuroimage.usc.edu/brainstorm/Introduction.

[18] ISO/IEC/IEEE 29148:2018, "Systems and software engineering — Life cycle processes — Requirements engineering,"2.ª ed., International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers, nov. 2018. [En línea]. Disponible: https://www.iso.org/standard/72089.html.

[19] P. Eeles, Çapturing Architectural Requirements,Rational Software Corporation, nov. 2001. [En línea]. Disponible: https://www.ibm.com/developerworks/rational/library/4706.html.

11 ANEXOS

11.1 MRCU - Matriz de Requerimientos y Casos de Uso

[Escriba aquí el contenido de esta sección.]

11.2 PMP — Project Management Plan

[Escriba aquí el contenido de esta sección.]

11.3 SAD - Software Architecture Document

[Escriba aquí el contenido de esta sección.]

11.4 SDD - Software Design Document

[Escriba aquí el contenido de esta sección.]

11.5 SRS - Software Requirements Specifications

[Escriba aquí el contenido de esta sección.]

11.6 TD - Test Document

[Escriba aquí el contenido de esta sección.]

11.7 MU - Manual de Usuario

[Escriba aquí el contenido de esta sección.]

11.8 VFP - Versión Final Propuesta de Proyecto

[Escriba aquí el contenido de esta sección.]

11.9 Aplicación de escritorio

[Escriba aquí el contenido de esta sección.]